

## Highlights

### **Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem**

Hernán Díaz Rodríguez

- Unified extreme critical path framework for neighbourhoods in the interval JSP.
- Feasibility, connectivity and no-loss proofs for any admissible interval ranking.
- LEX2 yields 5–11% narrower intervals by tightening the worst-case critical path.
- $N_2$  and  $N_8$  are the strongest neighbourhoods after irace-tuned tabu search;  $N_3$  worst.

# Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez<sup>a,\*</sup>

<sup>a</sup>*Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain*

---

## Abstract

The Job Shop Scheduling Problem with interval-valued processing times (IJSP) models real-world uncertainty where task durations are unknown but bounded. Comparing schedules then requires an interval ranking operator, and the classical notion of critical path must be extended to both extreme scenarios. I formalise *extreme critical paths* — paths critical in the best-case or worst-case scenario graph — and use them to define a neighbourhood  $H(\sigma)$  for local search. I prove that  $H(\sigma)$  is feasible, connected and loses no improving neighbours under any admissible interval ranking. Five neighbourhood variants ( $N_1$ ,  $N_2$ ,  $N_3$ ,  $N_{\text{ext}}$ ,  $N_8$ ) are formulated within this framework and combined with four ranking operators (EV, LEX1, LEX2, YX) on 82 benchmark instances of up to 1000 operations. Two findings are noteworthy: (i) after irace-based tabu search tuning,  $N_2$  and  $N_8$  emerge as the two strongest neighbourhoods (statistically tied, negligible effect  $r = 0.07$ ), both outperforming  $N_{\text{ext}}$  by a medium effect and  $N_1$  by a large effect;  $N_3$  is weakest by a large margin. (ii) LEX2 yields makespan intervals 5–11% narrower than the alternatives on average; a float analysis of both extreme graphs shows LEX2 solutions concentrate critical operations in the worst-case graph, consistent with LEX2 directly tightening the worst-case critical path. (iii) On the full 82-instance IJSP benchmark, the irace-tuned TS- $N_2$  reduces mean RE (%) by 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances relative to the previous best published solver; the neighbourhood ranking is consistent across all instance size classes.

*Keywords:* Job shop scheduling, combinatorial optimisation, local search,

---

\*Corresponding author

*Email address:* diazhernan@uniovi.es (Hernán Díaz Rodríguez)

## 1. Introduction

The Job Shop Scheduling Problem (JSP) is a flagship combinatorial optimisation problem in operations research, and neighbourhood-based local search has been its most competitive algorithmic paradigm for three decades [1, 2, 3]. The effectiveness of these methods rests almost entirely on neighbourhood design: the critical-block insight of [2] — that reversing interior arcs of a critical block cannot improve the makespan — reduces neighbourhood size by two-thirds without sacrificing solution quality, and the extended neighbourhood of [3] refines this further with a heads-and-tails viability filter. In practice, however, processing times are rarely known precisely at planning time: operator variability, machine condition, and material heterogeneity introduce uncertainty that deterministic models cannot capture.

A natural and tractable representation for such uncertainty is the *closed interval*  $[p^-, p^+]$ , bounding the minimum and maximum possible durations [4, 5]. The resulting *Interval Job Shop Scheduling Problem (IJSP)* raises two intertwined challenges: comparing schedules requires an *interval ranking operator* [6, 7], since the makespan is now an interval; and the critical-path concept that underpins efficient neighbourhood design must be reconsidered, because different paths may become critical under different realisations of the processing times [8, 9, 10].

### 1.1. Contributions

This paper makes the following contributions.

1. Formal definition of extreme criticality for IJSP (Section 4). I define a task or arc as *extreme critical* if it lies on a critical path of the best-case graph  $G^-(\sigma)$  or the worst-case graph  $G^+(\sigma)$ , and prove that this definition encompasses all necessarily critical tasks and is contained within the set of possibly critical tasks, making it a computationally tractable and theoretically sound approximation of the general criticality concept of [9].
2. Neighbourhood  $H(\sigma)$  based on extreme critical arcs (Section 5). I define  $H(\sigma)$  as the set of all feasible processing orders reachable by reversing a single extreme critical disjunctive arc, and prove three structural

properties: feasibility (every move yields a feasible order), no loss of improving neighbours (moves outside  $H(\sigma)$  cannot improve the makespan under any admissible ranking), and connectivity (every non-optimal order can reach a global optimum via a finite sequence of moves in  $H(\sigma)$ ).

3. Five neighbourhood variants within the  $H(\sigma)$  framework (Section 6). I formulate  $N_1$ ,  $N_2$ ,  $N_3$ ,  $N_{\text{ext}}$  and a new variant  $N_8$  (boundary arc swaps extended with extra-block reinsertion moves, after [11]) as instances of  $H(\sigma)$ , characterising their theoretical size and the cost of generating them.
4. Empirical study on 82 benchmark instances (Section 7). Three experimental phases are reported: (a) a  $5 \times 4$  comparison of four ranking operators at a common parameter setting (Section 7.2); (b) a head-to-head comparison of the five neighbourhoods after irace-based hyperparameter tuning, isolating the structural effect of the neighbourhood (Section 7.3); and (c) a comparison against the state of the art (Section 7.4), showing that the irace-tuned TS- $N_2$  achieves strictly better average solution quality than the previous best published solver.
5. Discovery and structural explanation of an interval-width effect (Section 7.2). One ranking operator consistently yields makespan intervals narrower than the alternatives. A float analysis of both extreme graphs reveals the structural mechanism: the operator concentrates critical operations in the worst-case graph  $G^+(\sigma)$ , directly tightening the worst-case critical path.

### 1.2. Paper Organisation

Section 2 provides background on the deterministic JSP and interval ranking operators. Section 3 reviews related work on neighbourhood structures and published IJSP solvers. Section 4 defines extreme criticality. Section 5 defines the neighbourhood  $H(\sigma)$  and proves its three structural properties. Section 6 describes the five neighbourhood variants. Section 7 reports the experimental study. Section 8 discusses the findings, and Section 9 concludes.

## 2. Background

### 2.1. The Deterministic Job Shop Problem

The JSP consists of  $n$  jobs  $\{J_1, \dots, J_n\}$  and  $m$  machines  $\{M_1, \dots, M_m\}$ . Each job  $J_i$  is a sequence of operations  $O_{i,1}, \dots, O_{i,m}$ , where operation  $O_{i,j}$

must be processed on machine  $M_j$  for exactly  $p_{i,j}$  time units. Operations within a job must respect their precedence order, and each machine processes at most one operation at a time. The goal is to find a *feasible processing order*  $\sigma$  minimising the makespan  $C_{\max}$ .

The *solution graph*  $G(\sigma) = (V, A \cup D(\sigma))$  represents a feasible order:  $V$  is the set of operations augmented with a dummy source node  $S$  and sink node  $T$ ;  $A$  contains precedence arcs within jobs; and  $D(\sigma)$  contains disjunctive arcs imposed by the order  $\sigma$  on each machine. The makespan equals the length of the longest path in  $G(\sigma)$ . The classical insight of [1] is that the neighbourhood for local search should restrict to reversals of arcs on critical paths, ensuring feasibility, while [2] shows that reversals of *interior* arcs of a critical block cannot improve the makespan, allowing further pruning.

## 2.2. Interval Arithmetic and Ranking Operators

When processing times are uncertain, I represent each as  $\mathbf{p} = [p^-, p^+] \in \mathbb{IR}$ , where  $\mathbb{IR}$  denotes the set of closed intervals. The IJSP requires three arithmetic operations on  $\mathbb{IR}$  [4]:

$$\mathbf{a} + \mathbf{b} = [a^- + b^-, a^+ + b^+], \quad (1)$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(a^-, b^-), \max(a^+, b^+)]. \quad (2)$$

Since there is no natural total order on  $\mathbb{IR}$ , comparing schedules requires choosing a *ranking operator*. For any interval  $\mathbf{a} = [a^-, a^+]$ , define its *midpoint*  $m(\mathbf{a}) = \frac{a^- + a^+}{2}$  and *width*  $w(\mathbf{a}) = a^+ - a^-$ . I consider four rankings widely used in the literature [6, 12, 5]:

$$\mathbf{a} \leq_{\text{LEX1}} \mathbf{b} \iff a^- < b^- \vee (a^- = b^- \wedge a^+ \leq b^+) \quad (3)$$

$$\mathbf{a} \leq_{\text{LEX2}} \mathbf{b} \iff a^+ < b^+ \vee (a^+ = b^+ \wedge a^- \leq b^-) \quad (4)$$

$$\mathbf{a} \leq_{\text{YX}} \mathbf{b} \iff m(\mathbf{a}) < m(\mathbf{b}) \vee (m(\mathbf{a}) = m(\mathbf{b}) \wedge w(\mathbf{a}) \leq w(\mathbf{b})) \quad (5)$$

$$\mathbf{a} \leq_{\text{EV}} \mathbf{b} \iff m(\mathbf{a}) \leq m(\mathbf{b}) \quad (6)$$

LEX1, LEX2 and YX are *admissible* total orders on  $\mathbb{IR}$  [6]: they refine the partial product order  $\leq_2$  defined by  $\mathbf{a} \leq_2 \mathbf{b} \iff a^- \leq b^- \wedge a^+ \leq b^+$ . EV ranks intervals by their midpoint  $m(\mathbf{a})$  and induces a total preorder rather than a strict total order, since two distinct intervals may share the same midpoint; it also refines  $\leq_2$ . This refinement property is essential for the theoretical guarantees of Section 5: any move that simultaneously decreases both  $C_{\max}^-$  and  $C_{\max}^+$  is an improvement under all four rankings considered here.

### 3. Related Work

*Neighbourhood structures for fuzzy and interval JSP..* The first neighbourhood for the fuzzy/interval JSP was proposed in [10], extending the van Laarhoven neighbourhood [1] to use possibly critical arcs rather than necessarily critical ones. This corresponds to what I call  $N_1$  in the present framework. Subsequent work introduced  $N_2$  as a filtered version of  $N_1$  that retains only block-boundary arcs [13], reducing neighbourhood size by  $\sim 64\%$  with statistically identical solution quality on the fuzzy JSP.  $N_3$  extends  $N_2$  with three-task rearrangements at block boundaries [14]. For the related fuzzy job shop, evolutionary tabu search combining these neighbourhood structures with genetic operators has been proposed for due-date satisfaction objectives [15]. Lexicographic goal-programming strategies have also been explored for the green fuzzy FJSP with vague production targets [16] and for the green interval FJSP combining makespan and energy objectives [17]. Prior approaches to the IJSP specifically have combined neighbourhood search with genetic algorithms for makespan minimisation [18] and tardiness [19], providing early evidence that the choice of ranking operator influences solution quality.

The Nowicki–Smutnicki extended neighbourhood [3], adapted here as  $N_{\text{ext}}$ , was originally proposed for the crisp JSP and achieves strong empirical performance by additionally considering interior block arcs that pass a heads-and-tails viability check. Its adaptation to the IJSP is a contribution of this work, exploiting the fact that the boundary-arc result of [2] can be lifted simultaneously to both extreme graphs  $G^-(\sigma)$  and  $G^+(\sigma)$ . A fifth variant,  $N_8$ , is also proposed here: it extends  $N_2$  with extra-block reinsertion moves that relocate critical block endpoints to positions outside the block, adapting the idea of [11] — originally developed for the crisp JSP — to the interval setting. For the deterministic JSP and FJSP, general frameworks for feasibility checking and quality evaluation of neighbourhood moves have been proposed in [20, 21]; analogous efficiency improvements for the interval setting remain an open direction. In a complementary line, machine learning methods have been proposed to estimate quantile-based risk measures of the IJSP makespan [22].

*Metaheuristic solvers for the IJSP..* The fast-elitist Artificial Bee Colony ( $ABC_{E3}$ ) [23] established best-known results on the 82-instance benchmark used here, combining ABC exploration with hill-climbing local search. The

Elitist-Selection ABC (*ESABC*) [24] further improved solution quality by incorporating a simulated-annealing diversity mechanism, at the cost of increased runtime. These two methods, together with the earlier GA of [18], constitute the state of the art against which the proposed tabu search is compared in Section 7.4.

## 4. Extreme Criticality for IJSP

### 4.1. Configurations and Extreme Scenario Graphs

A *configuration*  $\omega = (p_1, \dots, p_o)$ , where  $o = n \times m$ , assigns a deterministic duration  $p_i \in \mathbf{p}_i$  to each operation  $i$ ; let  $\Omega$  denote the set of all configurations. Given a feasible processing order  $\sigma$ , two *extreme scenario graphs* are of particular interest:

- $G^-(\sigma)$ : identical to  $G(\sigma)$  but with each arc  $(x, y)$  weighted by  $p_x^-$  (best case).
- $G^+(\sigma)$ : identical to  $G(\sigma)$  but with each arc  $(x, y)$  weighted by  $p_x^+$  (worst case).

The makespan interval satisfies

$$\mathbf{C}_{\max}(\sigma) = [C_{\max}^-(\sigma), C_{\max}^+(\sigma)], \quad (7)$$

where  $C_{\max}^-(\sigma)$  is the length of the longest path in  $G^-(\sigma)$  and  $C_{\max}^+(\sigma)$  is the length of the longest path in  $G^+(\sigma)$ . An illustrative example is shown in Figure 1: two jobs ( $J_1: O_{11}(M_1, [2, 6]) \rightarrow O_{12}(M_2, [2, 3])$ ;  $J_2: O_{21}(M_2, [4, 4]) \rightarrow O_{22}(M_1, [1, 4])$ ), with schedule  $\sigma$  fixing  $M_1$  order  $(O_{11}, O_{22})$  and  $M_2$  order  $(O_{21}, O_{12})$ . In  $G^-(\sigma)$  the critical path runs through the  $M_2$  disjunctive arc ( $C_{\max}^- = 6$ ); in  $G^+(\sigma)$  it runs through the  $M_1$  disjunctive arc ( $C_{\max}^+ = 10$ ). The critical arcs in the two extreme graphs differ, motivating the need to consider both.

### 4.2. Heads, Tails and Floats

For each extreme graph  $G^*(\sigma)$  ( $* \in \{-, +\}$ ), the *head*  $r_x^*$  of task  $x$  is the length of the longest path from the dummy start node to  $x$ , and the *tail*  $q_x^*$  is the length of the longest path from the completion of  $x$  to the end node. The *float* of  $x$  in  $G^*(\sigma)$  is

$$F_x^*(\sigma) = C_{\max}^*(\sigma) - r_x^* - p_x^* - q_x^*. \quad (8)$$

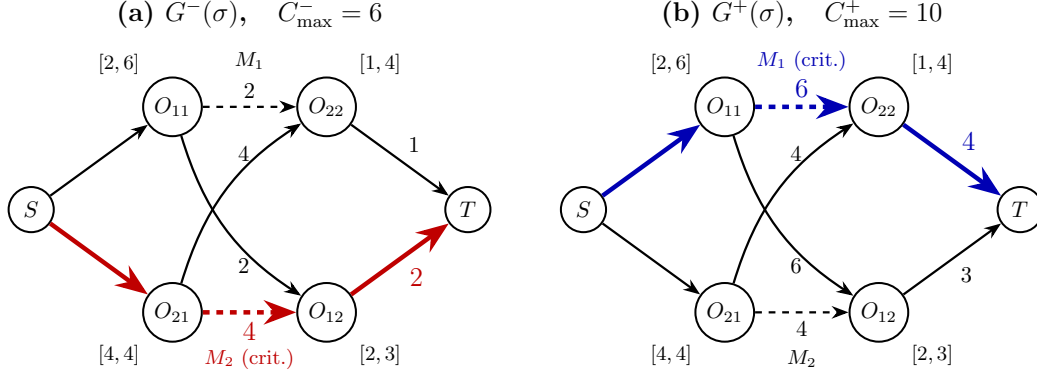


Figure 1: Illustrative example. (a) Best-case graph  $G^-(\sigma)$  with its critical path highlighted in red. (b) Worst-case graph  $G^+(\sigma)$  with its (different) critical path highlighted in blue. Precedence arcs are drawn solid, disjunctive arcs dashed.

A task is critical in  $G^*(\sigma)$  iff  $F_x^*(\sigma) = 0$ . The general problem of computing the minimum float  $\min_{\omega \in \Omega} F_x(\sigma, \omega)$  over all configurations — which equals zero iff  $x$  is possibly critical — is NP-hard [25, 26]; efficient branch-and-bound algorithms for this harder problem are given in [27]. The extreme-scenario floats  $F_x^-$  and  $F_x^+$  used here are computable in linear time and provide the tractable approximation exploited in Sections 5 and 6.

#### 4.3. Definition and Connection to Possibly/Necessarily Critical Tasks

**Definition 1** (Extreme critical path). *A path  $P$  in  $G(\sigma)$  is extreme critical if it is critical in  $G^-(\sigma)$  or in  $G^+(\sigma)$ . A node, arc or block is extreme critical if it lies on an extreme critical path.*

This definition connects to the general criticality concepts of [25, 9] as follows. A task  $x$  is *possibly critical* under  $\sigma$  if there exists  $\omega \in \Omega$  such that  $F_x(\sigma, \omega) = 0$ ; it is *necessarily critical* if  $F_x(\sigma, \omega) = 0$  for every  $\omega \in \Omega$  [25, 9].

**Proposition 1.** *Every extreme critical task is possibly critical. Every necessarily critical task is extreme critical.*

*Proof.* (i) *Extreme critical  $\Rightarrow$  possibly critical.* Let task  $x$  be extreme critical; then  $F_x^-(\sigma) = 0$  or  $F_x^+(\sigma) = 0$ . The extreme configurations  $\omega^- = (p_i^-)_i$  and  $\omega^+ = (p_i^+)_i$  both belong to  $\Omega$  (since  $p_i^\pm \in [p_i^-, p_i^+]$  by definition). In case  $F_x^-(\sigma) = 0$ : the float of  $x$  under configuration  $\omega^-$  equals  $F_x(\sigma, \omega^-) = C_{\max}^-(\sigma) - r_x^- - p_x^- - q_x^- = F_x^-(\sigma) = 0$ , so  $x$  lies on a critical path for  $\omega^- \in \Omega$ , making it possibly critical [25, 9]. The case  $F_x^+(\sigma) = 0$  is analogous with  $\omega^+$ .



(ii) *Necessarily critical  $\Rightarrow$  extreme critical.* If  $x$  is necessarily critical, then  $F_x(\sigma, \omega) = 0$  for every  $\omega \in \Omega$ . Taking  $\omega = \omega^- \in \Omega$  gives  $F_x^-(\sigma) = 0$ , so  $x$  is critical in  $G^-(\sigma)$ , hence extreme critical.  $\square$

The practical consequence is that extreme criticality offers a computationally tractable approximation to the exact criticality predicates: computing critical paths in  $G^-(\sigma)$  and  $G^+(\sigma)$  each costs  $O(|A \cup D(\sigma)|)$  via standard longest-path algorithms, while computing the full set of possibly critical tasks is NP-hard in general [25, 9].

## 5. The Neighbourhood $H(\sigma)$ and Its Properties

### 5.1. Definition

**Definition 2** (Neighbourhood  $H(\sigma)$ ). *Let  $\sigma_{(v)}$  denote the processing order obtained from  $\sigma$  by reversing a single disjunctive arc  $v \in D(\sigma)$ . The neighbourhood is*

$$H(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is extreme critical}\}. \quad (9)$$

*For comparison, the corresponding neighbourhood based on possibly critical arcs is*

$$H'(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is possibly critical}\}. \quad (10)$$

*By Proposition 1,  $H(\sigma) \subseteq H'(\sigma)$ , with strict inclusion in general.*

### 5.2. No Loss of Improving Neighbours

**Proposition 2.** *Let  $\sigma \in \Sigma$  be a feasible order and let  $v \in D(\sigma)$  be a disjunctive arc that is not extreme critical in  $G(\sigma)$ . Then*

$$C_{\max}^-(\sigma) \leq C_{\max}^-(\sigma_{(v)}) \quad \text{and} \quad C_{\max}^+(\sigma) \leq C_{\max}^+(\sigma_{(v)}), \quad (11)$$

*and consequently  $\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_{(v)})$  for every admissible ranking  $R$ .*

*Proof.* Fix  $*$   $\in \{-, +\}$ . Since  $(x, y)$  is not extreme critical, no critical path of  $G^*(\sigma)$  contains it; any such path, of length  $C_{\max}^*(\sigma)$ , remains a valid path in  $G^*(\sigma_{(v)})$  (whose only arc change is  $(x, y) \mapsto (y, x)$ ), hence  $C_{\max}^*(\sigma_{(v)}) \geq C_{\max}^*(\sigma)$ . Applying this to both extreme graphs gives  $\mathbf{C}_{\max}(\sigma) \leq_2 \mathbf{C}_{\max}(\sigma_{(v)})$ , and since every admissible  $R$  refines  $\leq_2$ , the claim follows.  $\square$

**Corollary 1.** *Neighbours in  $H'(\sigma) \setminus H(\sigma)$  can never improve the makespan under any of the four rankings considered.*

Corollary 1 justifies using  $H(\sigma)$  instead of the larger  $H'(\sigma)$ : the reduction is gained at zero cost in solution quality.

### 5.3. Feasibility

**Theorem 1** (Feasibility). *For every feasible  $\sigma \in \Sigma$ , the reversal of any extreme critical arc  $v = (x, y) \in D(\sigma)$  produces a feasible processing order  $\sigma_{(v)} \in \Sigma$ . Hence  $H(\sigma) \subseteq \Sigma$ .*

*Proof.* Suppose  $G(\sigma_{(v)})$  contains a directed cycle. Since  $G(\sigma)$  is acyclic and only  $(x, y)$  has been removed and  $(y, x)$  added, the cycle traverses  $(y, x)$ , yielding a directed path  $\pi$  from  $x$  to  $y$  in  $G(\sigma)$  not using  $(x, y)$ . In standard JSP,  $x$  and  $y$  lie on the same machine but in different jobs, so no precedence arc joins them;  $\pi$  has at least one intermediate operation  $u$ , and its length in  $G^*(\sigma)$  satisfies

$$\ell(\pi) = p_x^* + p_u^* + \cdots > p_x^*.$$

But  $(x, y)$  lies on a critical path of  $G^*(\sigma)$ , so the head of  $y$  is tight:  $r_y^* = r_x^* + p_x^*$ . The path  $\pi$  then gives  $r_y^* \geq r_x^* + \ell(\pi) > r_x^* + p_x^* = r_y^*$ , a contradiction.  $\square$

### 5.4. Connectivity

**Theorem 2** (Connectivity). *For every non-optimal  $\sigma \in \Sigma$ , there exists a finite sequence of moves in  $H(\sigma)$  leading to a globally optimal order  $\sigma^*$ .*

*Proof.* Fix any globally optimal  $\sigma^* \in \Sigma^*$  and define the *inversion count*

$$M(\sigma, \sigma^*) = |\{(x, y) \in D(\sigma) : (y, x) \in D(\sigma^*)\}|,$$

the number of machine-order disagreements;  $M = 0$  iff  $\sigma = \sigma^*$ .

*Key lemma.* If  $\lambda$  is non-optimal, there exists an extreme critical  $(x, y) \in D(\lambda)$  with  $(y, x) \in D(\sigma^*)$ .

*Proof.* Non-optimality implies  $\mathbf{C}_{\max}(\lambda) \not\leq_2 \mathbf{C}_{\max}(\sigma^*)$  (else  $\leq_R$  follows by refinement), so WLOG  $C_{\max}^-(\lambda) > C_{\max}^-(\sigma^*)$ . Let  $P$  be a critical path in  $G^-(\lambda)$ . If every disjunctive arc of  $P$  lay in  $D(\sigma^*)$ ,  $P$  would be a valid path in  $G^-(\sigma^*)$  of length  $> C_{\max}^-(\sigma^*)$ , a contradiction. So  $P$  contains some  $(x, y) \in D(\lambda) \setminus D(\sigma^*)$ ; since machine orientations are mutually exclusive,  $(y, x) \in D(\sigma^*)$ , and  $(x, y)$  is extreme critical by membership in  $P$ .  $\square_{\text{lemma}}$

*Construction.* Build  $\lambda_0 = \sigma$ ; while  $\lambda_k$  is non-optimal, apply the key lemma to obtain  $(x, y)$  and set  $\lambda_{k+1} = (\lambda_k)_{(x,y)} \in H(\lambda_k)$  (feasible by Theorem 1). Each reversal removes  $(x, y)$  from  $D(\lambda_k)$  (one fewer inversion) and adds  $(y, x)$  (already in  $D(\sigma^*)$ , no new inversion), so  $M(\lambda_{k+1}, \sigma^*) = M(\lambda_k, \sigma^*) - 1$ . Since  $M \geq 0$ , the loop terminates in at most  $M(\sigma, \sigma^*)$  steps at a globally optimal  $\lambda_k$ .  $\square$

## 6. Neighbourhood Variants for IJSP

I instantiate five concrete neighbourhood variants within the  $H(\sigma)$  framework, differing in which subset of extreme critical arcs each one considers. Within a critical block  $B = (o_1, \dots, o_k)$ , an arc  $(o_i, o_{i+1})$  is a *boundary arc* if  $i = 1$  or  $i = k - 1$ , and an *interior arc* otherwise. Figure 2 illustrates a critical block and the arcs included by each variant.

All five variants inherit the three structural guarantees of Section 5 (feasibility, no-loss of improving neighbours, and connectivity) for their arc-swap component, either directly (when only interior arcs are excluded, by Proposition 2) or by superset relation to a variant that does.  $N_8$  additionally introduces reinsertion moves that fall outside the framework; their feasibility is verified at runtime, and  $N_8$  is therefore included as an exploratory variant alongside the four arc-swap variants.

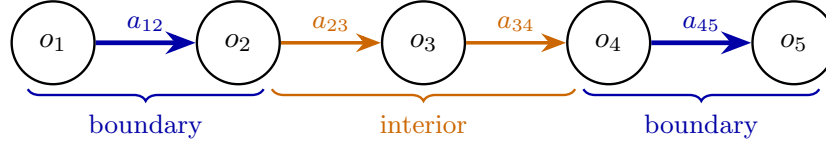
### 6.1. $N_1$ : All Extreme Critical Arcs

$N_1$  includes every disjunctive arc  $(x, y)$  such that  $x$  immediately precedes  $y$  on the same machine in  $G(\sigma)$  and the arc lies on a critical path of  $G^-(\sigma)$  or  $G^+(\sigma)$ . No positional filter is applied within the block: both boundary and interior arcs of every extreme critical block are candidates. Generation cost:  $O(|E|)$  per extreme graph.

### 6.2. $N_2$ : Block-Boundary Filter

$N_2$  restricts  $N_1$  to arcs at the *boundary* of each extreme critical block. A block  $B = \{o_1, \dots, o_k\}$  is a maximal contiguous subsequence of operations on the same machine that all lie on a single extreme critical path.  $N_2$  retains the two boundary arcs  $(o_1, o_2)$  and  $(o_{k-1}, o_k)$  and discards interior arcs  $(o_i, o_{i+1})$ ,  $2 \leq i \leq k - 2$ . This is the direct adaptation of [2] to the interval setting: by Proposition 2 applied simultaneously to  $G^-(\sigma)$  and  $G^+(\sigma)$ , the discarded interior arcs cannot carry improving moves under any of the four rankings, so  $N_2$  inherits the same theoretical guarantees as  $N_1$ .

Critical block  $B = (o_1, o_2, o_3, o_4, o_5)$  on machine  $M_k$



|                  | $a_{12}$ | $a_{23}$ | $a_{34}$ | $a_{45}$       |
|------------------|----------|----------|----------|----------------|
| $N_1$            | •        | •        | •        | •              |
| $N_2$            | •        | —        | —        | •              |
| $N_3$            | •        | •        | •        | • <sup>†</sup> |
| $N_{\text{ext}}$ | •        | $?_v$    | $?_v$    | •              |
| $N_8$            | •        | —        | —        | • <sup>‡</sup> |

• always included — always excluded  $?_v$  conditional on `isViableSwap` ( $N_{\text{ext}}$ )

<sup>†</sup>  $N_3$  additionally generates three rearrangement types for each boundary triple  $(x, y, z)$  of consecutive extreme-critical tasks on the same machine: cyclic left-shift  $(x, y, z \rightarrow y, z, x)$ , cyclic right-shift  $(x, y, z \rightarrow z, x, y)$ , and full reversal  $(x, y, z \rightarrow z, y, x)$ .

<sup>‡</sup>  $N_8$  additionally generates extra-block reinsertion moves for each block endpoint (see Section 6.5).

Figure 2: Critical block  $B = (o_1, \dots, o_5)$  on machine  $M_k$  and arcs considered by each neighbourhood variant.

### 6.3. $N_3$ : Subsequence Reversals

$N_3$  augments the  $N_2$  boundary arc swaps with three rearrangement types applied to *boundary triples* of consecutive extreme-critical tasks. For each triple  $(x, y, z)$  of successive operations on the same machine that are all extreme critical and span a block boundary (i.e.  $x$  is the first task in its block or  $z$  is the last),  $N_3$  generates three candidate moves: a cyclic left-shift  $(x, y, z \rightarrow y, z, x)$ , a cyclic right-shift  $(x, y, z \rightarrow z, x, y)$ , and a full reversal  $(x, y, z \rightarrow z, y, x)$ . Each candidate is checked for feasibility before being retained. The resulting neighbourhood strictly contains  $N_2$ .

### 6.4. $N_{\text{ext}}$ : Extended Nowicki–Smutnicki Neighbourhood

$N_{\text{ext}}$  adapts the extended neighbourhood of [3]. It starts from  $N_2$  (boundary arcs unconditionally) and additionally considers interior block arcs that pass a heads-and-tails viability check, denoted `isViableSwap`:

- Boundary arcs  $(o_1, o_2)$  and  $(o_{k-1}, o_k)$  are always included (as in  $N_2$ ).
- Interior arcs  $(o_i, o_{i+1})$  with  $2 \leq i \leq k - 2$  are included only if the `isViableSwap` check returns true.

The check `isViableSwap` $(x, y)$  computes an interval lower bound  $\mathbf{LB}(x, y)$  on the post-swap makespan  $\mathbf{C}_{\max}(\sigma_{(x,y)})$  by recalculating only the local heads and tails of  $x$  and  $y$  on both extreme graphs and combining the two per-operation estimates  $(\mathbf{r}_x + \mathbf{p}_x + \mathbf{q}_x$  and  $\mathbf{r}_y + \mathbf{p}_y + \mathbf{q}_y$ , all intervals) component-wise with interval max. The swap is discarded if  $\mathbf{LB}(x, y)$  is not better than the current incumbent under the active ranking. Because  $\mathbf{LB}(x, y) \leq_2 \mathbf{C}_{\max}(\sigma_{(x,y)})$  and every admissible ranking refines  $\leq_2$ , discarding is theoretically safe.

#### 6.5. $N_8$ : Block-Boundary Swaps with Extra-Block Reinsertion

$N_8$  is an exploratory variant that extends  $N_2$  with *extra-block reinsertion* moves, adapting the neighbourhood of [11] to the interval setting. For a critical block  $B = (o_1, \dots, o_k)$  on a machine, with machine predecessor  $o_0$  (if any) and machine successor  $o_{k+1}$  (if any),  $N_8$  includes all  $N_2$  boundary arc swaps and additionally considers four reinsertion candidates per block:

- move  $o_1$  to just after  $o_k$  (between  $o_k$  and  $o_{k+1}$ );
- move  $o_k$  to just before  $o_1$  (between  $o_0$  and  $o_1$ );
- move  $o_1$  to the head of the machine (before all tasks on that machine);
- move  $o_k$  to the tail of the machine (after all tasks on that machine).

Each reinsertion candidate is first filtered by a heads-and-tails estimate  $\mathbf{LB}_8$  of the post-reinsertion makespan, computed from cached heads and tails of the moved operation and its old and new machine neighbours (no graph propagation). The candidate is admitted unless  $\mathbf{LB}_8 \geq_2 \mathbf{C}_{\max}^{\text{best}}$ , i.e., dominated in both bounds. Admitted candidates are then evaluated exactly by applying the move and propagating head changes via breadth-first propagation through machine and job successors, with early termination if the running bound becomes worse than the incumbent under the active ranking.

The motivation is the empirical question of whether extra-block moves benefit interval scheduling. As shown in Section 7.3, the answer is search-strategy dependent:  $N_8$  underperforms under hill climbing (Phase A) but matches  $N_2$  under irace-tuned tabu search (Phase B); Section 8.4 explains why.

## 7. Experimental Study

The experimental study is organised in three phases that address distinct questions. Phase A (Section 7.2) compares the four ranking operators at a *common* hyperparameter setting, focusing on expected makespan (quality) and interval width (uncertainty). Phase B (Section 7.3) then fixes the ranking operator and gives each of the five neighbourhoods its own irace-tuned configuration, comparing the neighbourhoods on equal footing. Finally, Phase C (Section 7.4) compares the best Phase B configuration against the state of the art in IJSP, providing external validation of the proposed approach.

### 7.1. Setup Common to All Phases

*Instances..* The benchmark suite contains 82 instances split into two groups. The first group (*classical benchmarks*) comprises 12 instances selected from the OR-Library [28]: abz7, abz8, abz9 ( $20 \times 15$ ); ft10 ( $10 \times 10$ ), ft20 ( $20 \times 5$ ); la21, la24, la25 ( $15 \times 10$ ), la27, la29 ( $20 \times 10$ ), la38, la40 ( $15 \times 15$ ). The second group comprises 70 instances from the Taillard benchmark [29], covering seven size classes with 10 instances each:  $15 \times 15$  (225 operations),  $20 \times 15$  (300),  $20 \times 20$  (400),  $30 \times 15$  (450),  $30 \times 20$  (600),  $50 \times 15$  (750) and  $50 \times 20$  (1 000). All 82 instances are derived from their deterministic counterparts by perturbing each nominal processing time  $p_{ij}^*$  with a  $\pm 7.5\%$  uniform symmetric interval,  $\mathbf{p}_{ij} = [0.925 p_{ij}^*, 1.075 p_{ij}^*]$  (15% relative width).

*Hardware and software..* All experiments were executed on an Intel Xeon E5-2680 v4 workstation (14 cores / 28 threads, 2.40 GHz, 16 GB RAM) running Windows 10 with a WSL2 Linux environment, dispatching up to 24 processes in parallel (one per thread). The fEABC algorithm was implemented in C++14 and compiled with GCC; statistical analysis was performed in R.

*Algorithm..* All configurations use *fEABC* [23], a population-based hybrid metaheuristic that combines an Artificial Bee Colony (ABC) framework, Particle Swarm Optimisation (PSO) velocity updates for solution generation, and an embedded local search applied at each generation to a fraction of the population, with Lamarckian propagation of improvements. Solutions are encoded as permutations with repetition (*job-order*) and decoded via an insertion-based Schedule Generation Scheme (SGS). The local search component differs by phase: Phase A uses first-improvement hill climbing as a neutral, operator-agnostic baseline; Phase B uses a tabu search (best-neighbour selection with a short-term tabu list) independently tuned by irace

for each neighbourhood (Section 7.3; the rationale for this switch is discussed there). The stopping criterion differs by phase: Phase A terminates after 25 consecutive generations without improvement in the global best; Phase B imposes an additional wall-clock limit of 900 s (15 minutes) per run. Thirty independent runs are executed for every (instance, configuration) pair.

*Hyperparameter configuration..* Phase A adopts the configuration validated in the fEABC paper [23]: JOX (Job Order Crossover), inversion mutation, the food-source abandonment limit  $NT_{\max} = 20$  (a candidate solution not improved after  $NT_{\max}$  trials is discarded and reinitialised), a population of 250 with elite size 40, and first-improvement hill-climbing local search applied to 75% of individuals per generation.

Phase B replaces hill climbing with tabu search and tunes six parameters independently per neighbourhood using irace [30] (R 4.5.3): crossover operator, mutation operator, elite size,  $NT_{\max}$ , local-search target fraction, and the bad-iteration limit (maximum consecutive non-improving tabu iterations). The tuning used a budget of 1 500 candidate evaluations per neighbourhood and a Friedman-test racing criterion. The remaining parameters are fixed: population 250, tabu-list minimum size 3, and LEX2 ranking (selected in Phase A; Section 7.2). Table 1 reports the resulting configurations. The subsequent Phase B evaluation comprised 12 300 runs (five neighbourhoods  $\times$  82 instances  $\times$  30 runs); tuning and evaluation together required approximately nine days of wall-clock time.

Table 1: Irace-tuned hyperparameter configurations for Phase B. Parameters not listed are fixed: population 250, tabu-min-size 3, LEX2 ranking. *LS target*: fraction of individuals receiving local search per generation. *Bad-iter*: maximum consecutive non-improving tabu iterations before the local search terminates.

|                  | Crossover | Mutation  | Elite | $NT_{\max}$ | LS target | Bad-iter |
|------------------|-----------|-----------|-------|-------------|-----------|----------|
| $N_1$            | JOX       | insertion | 33    | 11          | 85.3%     | 16       |
| $N_2$            | JOX       | insertion | 50    | 24          | 100%      | 20       |
| $N_3$            | GOX       | swap      | 48    | 25          | 82.8%     | 30       |
| $N_{\text{ext}}$ | JOX       | swap      | 39    | 17          | 76.2%     | 23       |
| $N_8$            | JOX       | swap      | 39    | 27          | 100%      | 21       |

*Statistics..* Comparisons use the Friedman test (non-parametric repeated-measures ANOVA; the statistic follows a  $\chi^2$  distribution with  $k - 1$  degrees of freedom for  $k$  groups) followed by pairwise Wilcoxon signed-rank tests with

Holm–Bonferroni correction. Effect size is reported as  $r = |Z|/\sqrt{N}$ , where  $Z$  is the standardised Wilcoxon test statistic and  $N$  is the number of paired observations, with the conventional thresholds (small  $\geq 0.1$ , medium  $\geq 0.3$ , large  $\geq 0.5$ ). Each (instance, run) pair forms a paired block. The metric is the midpoint of the makespan interval,  $(C_{\max}^- + C_{\max}^+)/2$ .

## 7.2. Phase A: Operator Calibration

Phase A is preparatory rather than central: its purpose is to identify the ranking operator that minimises makespan interval uncertainty at negligible midpoint cost, so that it can be fixed for the neighbourhood comparison in Phase B. The  $5 \times 4$  design (five neighbourhoods, four operators, common first-improvement hill-climbing local search) gives 49,200 runs and also establishes a neutral baseline.

*Hardware calibration..* To calibrate this cluster against the hardware used in the published IJSP solvers compared in Section 7.4, I ran the fEABC+HC configuration used here—identical in algorithm and parameters to the one reported in [23], which used an Intel Xeon Gold 6132 at 2.60 GHz under native Linux—on the same 12 classical instances and compared average runtimes: the ratio is approximately  $2 \times$  (geometric mean 2.10, range 1.4–2.8), indicating this cluster runs the identical workload roughly twice as slow. Runtime figures in Section 7.4 should be interpreted accordingly; RE (%) comparisons are hardware-independent.

*Midpoint quality..* I begin by comparing the four operators on solution quality, taken as the makespan-interval midpoint (lower is better). The Friedman test across operators yields  $\chi^2(3) = 694.96$ ,  $p < 10^{-150}$ , confirming that the operators are not all equivalent. Table 2 reports the global mean midpoint and median runtime; pairwise tests (Table 3) show that LEX1 is significantly worse than the other three (medium-to-large effects), EV and LEX2 are statistically indistinguishable ( $p = 0.197$ ,  $r = 0.026$ ), and YX leads on average but its advantage over EV and LEX2 carries only small effects ( $r \leq 0.131$ ). The choice of operator has no meaningful effect on wall-clock time: Kruskal–Wallis tests return  $p > 0.48$  for four of the five neighbourhoods, and the only significant case ( $N_{\text{ext}}$ ,  $p = 0.005$ ) involves absolute differences of  $\leq 52$ s.

*Interval width and its mechanism..* The width  $C_{\max}^+ - C_{\max}^-$  measures solution uncertainty [4, 5]. Table 4 shows that LEX2 yields intervals 5.3% narrower than EV and 10.7% narrower than LEX1; the ordering  $\text{LEX2} \prec \text{YX} \prec \text{EV}$



Table 2: Phase A: mean midpoint (mid.) and median wall-clock time  $t$ (s) per run, by neighbourhood and ranking operator (82 instances  $\times$  30 runs). For midpoint, lower is better.

| Neighbourhood    | EV     |     | LEX1   |     | LEX2   |     | YX     |     |
|------------------|--------|-----|--------|-----|--------|-----|--------|-----|
|                  | mid.   | $t$ | mid.   | $t$ | mid.   | $t$ | mid.   | $t$ |
| $N_1$            | 1888.9 | 52  | 1894.6 | 33  | 1889.5 | 37  | 1887.0 | 32  |
| $N_2$            | 1888.7 | 29  | 1895.2 | 41  | 1889.2 | 33  | 1887.1 | 29  |
| $N_3$            | 1907.3 | 194 | 1914.2 | 206 | 1908.5 | 254 | 1904.9 | 201 |
| $N_8$            | 1890.5 | 30  | 1894.6 | 33  | 1891.3 | 36  | 1887.5 | 31  |
| $N_{\text{ext}}$ | 1889.0 | 33  | 1891.0 | 85  | 1895.5 | 65  | 1887.2 | 33  |

Table 3: Phase A: pairwise Wilcoxon tests for ranking operators ( $n = 2460$  paired blocks, Holm–Bonferroni corrected).

| A    | B    | Mean A | Mean B | Diff | $p_{\text{adj}}$ | $r$   | Magnitude  |
|------|------|--------|--------|------|------------------|-------|------------|
| LEX1 | YX   | 1909.5 | 1901.5 | +8.0 | $< 10^{-3}$      | 0.509 | large      |
| EV   | LEX1 | 1903.3 | 1909.5 | −6.2 | $< 10^{-3}$      | 0.415 | medium     |
| LEX1 | LEX2 | 1909.5 | 1903.9 | +5.6 | $< 10^{-3}$      | 0.349 | medium     |
| LEX2 | YX   | 1903.9 | 1901.5 | +2.4 | $< 10^{-3}$      | 0.159 | small      |
| EV   | YX   | 1903.3 | 1901.5 | +1.8 | $< 10^{-3}$      | 0.131 | small      |
| EV   | LEX2 | 1903.3 | 1903.9 | −0.6 | 0.197            | 0.026 | negligible |

$\prec$  LEX1 holds across every neighbourhood. The effect is consistent at the instance level: LEX2 produces narrower intervals than LEX1 in 88.8%, than EV in 72.2%, and than YX in 70.5% of (instance, neighbourhood) pairs. Figure 3(a) confirms the distributional shift; panels (b) and (c) reveal its mechanism. The natural hypothesis is that LEX2 leaves more structural slack in  $G^+(\sigma)$ , letting the worst-case path shorten. The float analysis refutes this: LEX2 solutions have *more* critical operations and *less* per-operation float in  $G^+(\sigma)$ , the opposite of the slack hypothesis. The mechanism is directional: LEX2 accepts moves that reduce  $C_{\max}^+$  even when  $C_{\max}^-$  stays unchanged, directly tightening the upper-bound graph. This asymmetry between the two extreme graphs is LEX2’s structural fingerprint; Section 8.2 provides the theoretical account.

*Conclusion and transition to Phase B.* Although YX attains the best mean midpoint, its edge over LEX2 is small ( $r \leq 0.16$ ) and LEX2 is statistically equivalent to EV ( $p = 0.197$ ); since LEX2 also yields the narrowest intervals (5–11% narrower than the alternatives), all Phase B configurations fix LEX2.

Table 4: Mean makespan interval width by ranking operator (Phase A; mean over 5 neighbourhoods  $\times$  82 instances; one best-of-30 solution per configuration).

| Operator    | Mean width   | vs. EV       |
|-------------|--------------|--------------|
| EV          | 269.3        | —            |
| LEX1        | 286.4        | +6.4%        |
| <b>LEX2</b> | <b>255.0</b> | <b>−5.3%</b> |
| YX          | 267.5        | −0.7%        |

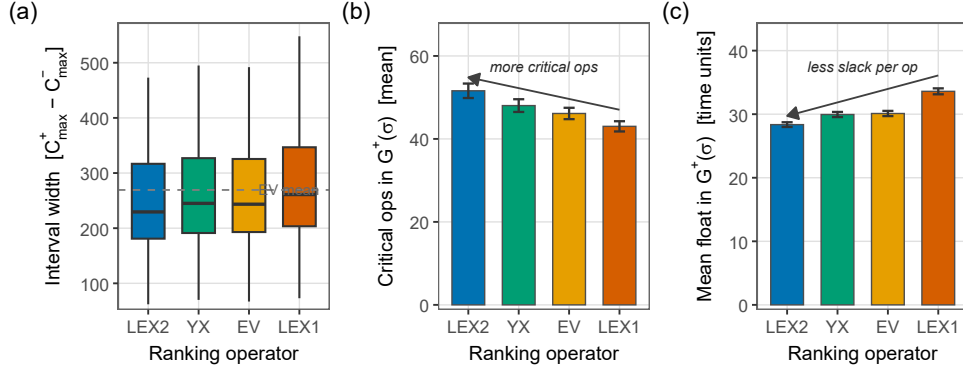


Figure 3: Phase A: operator comparison (5 neighbourhoods  $\times$  82 instances = 410 observations per operator; one best-of-30 solution per configuration). (a) Distribution of makespan interval width  $C_{\max}^+ - C_{\max}^-$ ; the dashed line marks the EV mean. (b) Mean number of critical operations in  $G^+(\sigma)$  (operations with  $F^+ = 0$ ). (c) Mean per-operation float in  $G^+(\sigma)$ . Panels (b) and (c) refute the slack hypothesis: LEX2 has *more* critical operations and *less* per-operation slack in  $G^+(\sigma)$ , confirming directional tightening of the worst-case critical path.

This is consistent with prior analyses where, even under per-operator hyperparameter tuning, LEX2 attained comparable solution quality with greater robustness [24, 32]; the relationship between robustness and interval width (uncertainty) remains to be characterised.

### 7.3. Phase B: Neighbourhood Comparison after Tuning

Phase B is the central experimental contribution. Each neighbourhood is given its own irace-tuned tabu search configuration (Table 1), isolating structural neighbourhood differences from confounding parameter choices. The switch from hill climbing to tabu search is necessary: hill climbing converges quickly to a local optimum and then stalls, suppressing any structural difference between neighbourhoods; tabu search escapes local optima through-

Table 5: Phase B: pairwise Wilcoxon tests for tuned neighbourhoods ( $n = 2460$  paired blocks, Holm–Bonferroni corrected). All differences are statistically significant; for the three  $N_3$  comparisons the  $p$ -value underflows double precision, so a representable bound is shown.

| A                | B                | Mean A | Mean B | Diff  | $p_{\text{adj}}$ | $r$   | Magnitude  |
|------------------|------------------|--------|--------|-------|------------------|-------|------------|
| $N_2$            | $N_3$            | 1846.5 | 1895.5 | −49.0 | $< 10^{-300}$    | 0.825 | large      |
| $N_3$            | $N_8$            | 1895.5 | 1847.8 | +47.7 | $< 10^{-300}$    | 0.824 | large      |
| $N_3$            | $N_{\text{ext}}$ | 1895.5 | 1853.8 | +41.7 | $< 10^{-300}$    | 0.794 | large      |
| $N_1$            | $N_3$            | 1868.6 | 1895.5 | −26.9 | $< 10^{-249}$    | 0.682 | large      |
| $N_1$            | $N_2$            | 1868.6 | 1846.5 | +22.1 | $< 10^{-222}$    | 0.643 | large      |
| $N_1$            | $N_8$            | 1868.6 | 1847.8 | +20.9 | $< 10^{-196}$    | 0.603 | large      |
| $N_1$            | $N_{\text{ext}}$ | 1868.6 | 1853.8 | +14.9 | $< 10^{-122}$    | 0.476 | medium     |
| $N_{\text{ext}}$ | $N_2$            | 1853.8 | 1846.5 | +7.3  | $< 10^{-61}$     | 0.337 | medium     |
| $N_{\text{ext}}$ | $N_8$            | 1853.8 | 1847.8 | +6.0  | $< 10^{-38}$     | 0.267 | small      |
| $N_2$            | $N_8$            | 1846.5 | 1847.8 | −1.3  | 0.001            | 0.070 | negligible |

out the full time budget, allowing each neighbourhood’s block geometry and candidate-set size to influence the search trajectory. Using the irace-tuned configurations, I evaluate the five neighbourhoods on the 82 benchmark instances (30 runs per instance,  $5 \times 82 \times 30 = 12,300$  observations).

Figure 4(a) shows the per-instance relative error RE (%):

$$\text{RE (\%)} = \frac{E[\mathbf{C}_{\text{max}}] - \text{LB}}{\text{LB}} \times 100, \quad (12)$$

where LB is the published lower bound of the corresponding *crisp* (deterministic) instance [29, 28, 23]. Since this bound need not coincide with the interval-makespan midpoint, RE (%) is an indicative common yardstick rather than a strict optimality gap; it is used uniformly throughout Sections 7.3–7.4. The Friedman test yields  $\chi^2(4) = 4120.58$ ,  $p < 10^{-150}$ ; mean ranks are  $N_2$ : 2.143,  $N_8$ : 2.245,  $N_{\text{ext}}$ : 2.638,  $N_1$ : 3.419,  $N_3$ : 4.555. Pairwise tests (Table 5) reveal that  $N_2$  and  $N_8$  are essentially tied (negligible effect,  $r = 0.07$ ,  $p = 0.001$ ), while both are separated from  $N_{\text{ext}}$  by a medium effect, from  $N_1$  by a large effect, and from  $N_3$  by a very large margin.

Figure 4(b) visualises the quality–runtime trade-off aggregated across instance groups:  $N_2$  achieves the lowest mean RE (%) at moderate runtime, with  $N_8$  nearly indistinguishable in quality at slightly higher cost;  $N_1$  is the fastest but sits at substantially worse quality (a large effect separates it from  $N_2$ ).

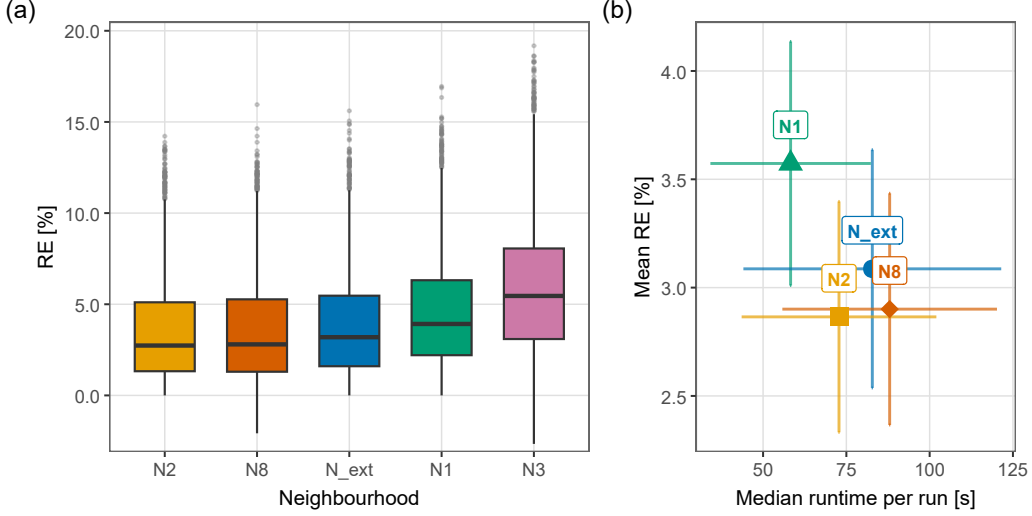


Figure 4: Phase B (irace-tuned tabu search). **(a)** Per-instance RE (%) on the makespan midpoint by neighbourhood (Eq. 12). Each box pools  $82 \times 30 = 2,460$  observations; pairwise significance and effect sizes are reported in Table 5. **(b)** Quality–runtime trade-off (N3 omitted for scale clarity): each point is the mean RE (%) vs. mean median runtime aggregated over all instance groups; error bars are  $\pm 1$  SE. Aggregated over all groups,  $N_2$  achieves the lowest mean RE (%) at moderate runtime ( $\approx 73$  s);  $N_8$  is slightly slower ( $\approx 88$  s) with nearly identical RE (%);  $N_1$  is fastest ( $\approx 58$  s) but incurs a large-effect quality penalty;  $N_{\text{ext}}$  sits between  $N_8$  and  $N_1$  on both axes.

Decomposing by instance group (Table 6),  $N_2$  posts the lowest group-level RE (%) on 7 of 8 groups—the aggregated Classical row and all Taillard size classes except tai50 $\times$ 15, where  $N_8$  leads.  $N_{\text{ext}}$ ,  $N_1$  and  $N_3$  never win at the group level; per-instance results (where  $N_2$  wins 54 of the 82 individual instances,  $N_8$  wins 22, and  $N_{\text{ext}}$  wins 5 — including LA24 and LA38) are provided in the supplementary material [31]. On runtime, the picture varies by instance size:  $N_1$  is the fastest neighbourhood on the small and medium Taillard classes (tai30 $\times$ 20: 102 s vs  $N_2$ ’s 170 s) but trails on quality by a large effect;  $N_{\text{ext}}$  becomes the *slowest* viable neighbourhood on the largest instances (tai50 $\times$ 15: 354 s; tai50 $\times$ 20: 695 s), exceeding  $N_2$ ’s runtimes (181 s; 554 s) by a factor of  $\approx 2$  at inferior quality.  $N_3$  again occupies the worst position on both dimensions: its median runtime reaches 898 s at tai50 $\times$ 20—effectively hitting the 900 s time limit—with the worst quality on every instance group.

Considered jointly,  $N_2$  offers the best efficiency profile across all instance

Table 6: Phase B: mean RE (%) and median runtime  $t$  (s) per instance group and neighbourhood (irace-tuned TS, 30 runs per instance). Bold: minimum RE (%) in each row. The Classical row aggregates the 12 OR-Library instances; Taillard rows aggregate the 10 instances of each size class. Per-instance results are provided as supplementary material [31].

| Group             | $n$ | $N_2$       |     | $N_8$       |     | $N_{\text{ext}}$ |     | $N_1$ |     | $N_3$ |     |
|-------------------|-----|-------------|-----|-------------|-----|------------------|-----|-------|-----|-------|-----|
|                   |     | RE          | $t$ | RE          | $t$ | RE               | $t$ | RE    | $t$ | RE    | $t$ |
| Classical         | 12  | <b>2.40</b> | 18  | 2.41        | 23  | 2.52             | 14  | 2.86  | 12  | 3.85  | 189 |
| 15×15             | 10  | <b>2.04</b> | 19  | 2.05        | 25  | 2.24             | 14  | 2.38  | 11  | 3.05  | 384 |
| 20×15             | 10  | <b>3.58</b> | 42  | 3.66        | 54  | 3.85             | 32  | 4.28  | 24  | 5.50  | 686 |
| 20×20             | 10  | <b>5.34</b> | 58  | 5.52        | 74  | 5.69             | 44  | 5.86  | 33  | 6.96  | 894 |
| 30×15             | 10  | <b>3.68</b> | 103 | 3.88        | 134 | 4.11             | 96  | 5.00  | 72  | 7.12  | 896 |
| 30×20             | 10  | <b>8.62</b> | 161 | 8.78        | 214 | 9.07             | 122 | 9.95  | 100 | 12.08 | 580 |
| 50×15             | 10  | 0.32        | 173 | <b>0.22</b> | 206 | 0.61             | 345 | 1.48  | 279 | 2.50  | 889 |
| 50×20             | 10  | <b>2.03</b> | 535 | 2.11        | 589 | 2.82             | 680 | 4.63  | 388 | 6.75  | 899 |
| <i>Grand mean</i> |     | 3.47        | 60  | 3.55        | 80  | 3.83             | 46  | 4.51  | 37  | 5.93  | 731 |

sizes: it ties with  $N_8$  for the best quality and is the faster of the two for small and medium instances, while being faster than  $N_{\text{ext}}$  on all instances with  $\geq 450$  operations.  $N_8$  is equally recommended when the instance is large and runtimes exceeding 500s are acceptable, particularly for the tai50×15 class where it achieves the lowest RE (%) (0.22% vs  $N_2$ 's 0.32%). The practical implication is that  $N_2$  is the default choice for most settings;  $N_8$  may be a viable alternative in some cases, depending on instance size and the available time budget.

### 7.3.1. Convergence Profiles

Figure 5 shows the per-generation convergence profiles of the five neighbourhoods, each under its irace-tuned configuration, averaged over all 82 instances and 30 runs. Each run's best midpoint is divided by its generation-0 value, so every curve starts at 1.0 and decreases. The bulk of the improvement is captured very early: all curves cover  $\sim 83$ – $86\%$  of their total reduction by generation 5 and  $\sim 90$ – $93\%$  by generation 10, and the neighbourhood ranking is already set from the first few generations. By generation 25 the curves have largely stabilised:  $N_2$  and  $N_8$  are essentially tied at the lowest values (0.803 and 0.802 respectively),  $N_{\text{ext}}$  sits at 0.807,  $N_1$  at 0.815, and  $N_3$  at 0.830, consistent with the statistical lead of  $N_2$  and  $N_8$  in Table 5.

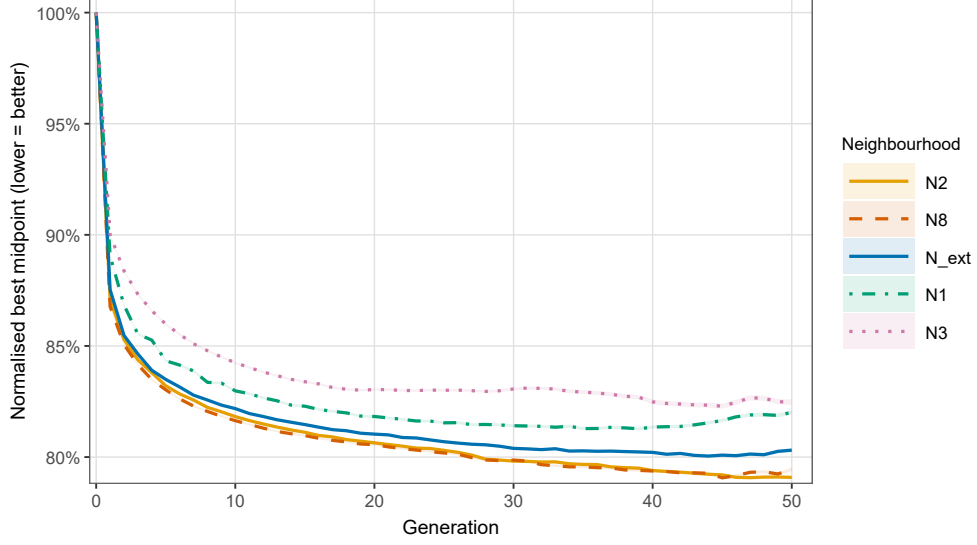


Figure 5: Per-generation normalised best-midpoint profiles, by neighbourhood (Phase B, irace-tuned tabu search; mean over 82 instances  $\times$  30 runs per neighbourhood; each run is normalised by its generation-0 value). The shaded ribbons represent  $\pm 1$  standard error.

#### 7.4. Comparison with Published IJSP Solvers

I compare the irace-tuned TS under neighbourhood  $N_2$  (Phase B best, Section 7.3) against three published IJSP metaheuristics on the same benchmark instances: a Genetic Algorithm (GA) [18], a fast-elitist Artificial Bee Colony ( $ABC_{E3}$ ) [23], and an Elitist-Selection ABC ( $ESABC$ ) [24]. All methods use 30 independent runs per instance; results for the published solvers are taken directly from the cited papers. The comparison metric is RE (%) (Equation 12), where the LB values are from [23] for the classical instances and from [29] for the Taillard instances.

*Classical instances.* Table 7 compares all four solvers on the 12 classical instances (ABZ7–9, FT10, FT20, LA21/24/25/27/29/38/40). TS- $N_2$  achieves a mean RE of **2.40%**, against 5.53% for ESABC, 6.39% for  $ABC_{E3}$ , and 9.78% for GA, a reduction of 56.6% relative to the previous best (ESABC). TS- $N_2$  attains the lowest mean RE on all 12 instances individually; the largest absolute gains arise on FT20 (0.23% vs. 1.78% for ESABC) and la27 (1.69% vs. 4.12%).

Table 7: Comparison with published IJSP solvers on 12 classical instances (Phase B, irace-tuned TS,  $N_2$ ). Metric:  $\text{RE}(\%) = (E[\mathbf{C}_{\max}] - \text{LB}) / \text{LB} \times 100$  (Eq. 12). LBs from [23]. GA from [18];  $ABC_{E3}$  from [23];  $ESABC$  from [24]. All methods: 30 independent runs. Bold: row-minimum average RE.  $t$  = median runtime (s).

| Instance    | LB   | $GA$ [18] |       |     | $ABC_{E3}$ [23] |       |     | $ESABC$ [24] |       |     | $TS-N_2$ (ours) |             |     |
|-------------|------|-----------|-------|-----|-----------------|-------|-----|--------------|-------|-----|-----------------|-------------|-----|
|             |      | Best      | Avg   | $t$ | Best            | Avg   | $t$ | Best         | Avg   | $t$ | Best            | Avg         | $t$ |
| ABZ7        | 656  | 6.33      | 12.50 | 1.8 | 5.26            | 7.32  | 4.5 | 4.19         | 6.73  | 6.5 | 1.83            | <b>3.03</b> | 32  |
| ABZ8        | 645  | 11.32     | 18.48 | 1.8 | 8.99            | 12.06 | 4.2 | 8.68         | 10.95 | 7.7 | 4.50            | <b>6.64</b> | 39  |
| ABZ9        | 661  | 13.01     | 17.97 | 2.2 | 9.68            | 13.13 | 6.0 | 8.55         | 11.19 | 8.7 | 5.07            | <b>6.31</b> | 40  |
| FT10        | 930  | 1.83      | 5.23  | 0.5 | 1.08            | 4.11  | 1.6 | 0.59         | 3.01  | 1.8 | 0.59            | <b>0.99</b> | 7   |
| FT20        | 1165 | 1.46      | 4.35  | 0.7 | 0.69            | 1.73  | 2.7 | 0.69         | 1.78  | 2.9 | 0.09            | <b>0.23</b> | 9   |
| la21        | 1046 | 3.15      | 5.01  | 1.1 | 2.58            | 5.01  | 1.8 | 2.06         | 3.96  | 2.9 | 0.57            | <b>1.39</b> | 11  |
| la24        | 935  | 4.06      | 6.34  | 0.8 | 2.25            | 5.06  | 2.7 | 3.53         | 4.95  | 2.6 | 0.80            | <b>1.37</b> | 10  |
| la25        | 977  | 1.94      | 5.11  | 1.0 | 1.94            | 3.88  | 2.4 | 1.33         | 2.74  | 3.0 | 0.20            | <b>1.31</b> | 10  |
| la27        | 1235 | 4.57      | 10.22 | 1.3 | 2.75            | 4.66  | 4.1 | 2.75         | 4.12  | 5.8 | 0.97            | <b>1.69</b> | 19  |
| la29        | 1152 | 11.11     | 14.23 | 1.1 | 5.51            | 8.65  | 4.4 | 4.99         | 7.03  | 6.7 | 1.95            | <b>2.87</b> | 21  |
| la38        | 1196 | 6.02      | 9.16  | 1.4 | 4.52            | 6.88  | 6.0 | 3.01         | 5.83  | 7.2 | 0.71            | <b>1.91</b> | 20  |
| la40        | 1222 | 5.07      | 8.74  | 1.2 | 1.88            | 4.21  | 3.0 | 2.74         | 4.11  | 3.7 | 0.70            | <b>1.09</b> | 17  |
| <i>Mean</i> |      | 9.78      |       |     | 6.39            |       |     | 5.53         |       |     | <b>2.40</b>     |             |     |

*Taillard instances..* Per-instance results on all 70 Taillard instances (TA1–TA70, seven size classes from  $15 \times 15$  to  $50 \times 20$ ) are provided as supplementary material [31]. GA results are unavailable for the Taillard IJSP instances; the comparison covers  $ABC_{E3}$  and ESABC only;  $TS-N_8$  is included alongside  $TS-N_2$  for reference.  $TS-N_2$  achieves a grand mean RE of **3.66%** across all 70 instances, against 8.66% for ESABC and 9.39% for  $ABC_{E3}$ , a 57.7% reduction relative to ESABC.  $TS-N_2$  achieves the strictly lower average RE than both ABC methods on every one of the 70 instances. The improvement is consistent across size classes and most pronounced on tai50 $\times$ 15 (TA51–TA60; group mean:  $TS-N_2 = 0.32\%$  vs.  $ESABC = 4.55\%$ ), where the interval TS nearly matches the lower bound—TA51 achieves  $\text{RE} = 0.02\%$  and TA53 achieves  $\text{RE} = 0.04\%$ . The hardest class, tai30 $\times$ 20 (TA41–TA50), yields a group mean of 8.62% for  $TS-N_2$  versus 15.32% for ESABC; these instances remain the most challenging for all methods.  $TS-N_8$  finishes marginally behind  $TS-N_2$  in grand mean (3.74% vs. 3.66%), consistent with the Phase B ranking (Section 7.3), though it outperforms  $N_2$  on individual instances where a single dominant critical path prevails (Section 8.4).

## 8. Discussion

### 8.1. Practical Recommendations

The experimental results lead to a small set of actionable recommendations:

1. Choose between  $N_2$  and  $N_8$  based on instance size and time budget.  $N_2$  is the default recommendation: it ties with  $N_8$  for the best quality overall (negligible difference,  $r = 0.07$ ) and is faster for small and medium instances (Table 6). On the tai50×15 class,  $N_8$  achieves the strictly lower group mean RE (%) (0.22% vs  $N_2$ 's 0.32%) at a moderate extra runtime (218 s vs 181 s), consistent with its structural advantage in the single-dominant-critical-path regime [11]; for tai50×20,  $N_2$  recovers the edge (2.03% vs  $N_8$ 's 2.11%, 554 s vs 603 s).  $N_8$  is thus the preferred choice when quality is paramount on the largest 50-job instances with 15 machines, while  $N_2$  is more robust across the full range.
2. Do not assume  $N_{\text{ext}}$  is best on the basis of Phase A results. Under irace-tuned tabu search,  $N_{\text{ext}}$  finishes third—a medium effect behind both  $N_2$  and  $N_8$ —and becomes the *slowest* viable neighbourhood on large instances (tai50×20: 695 s vs  $N_2$ 's 554 s and  $N_8$ 's 603 s, at a 0.79 pp quality cost; Table 6). The heads-and-tails viability filter that makes  $N_{\text{ext}}$  efficient under hill climbing becomes a runtime bottleneck under tabu search with a full neighbourhood evaluation per iteration.
3. Use LEX2 if interval width is the goal; operator choice has limited impact on midpoint. LEX2 yields makespan intervals 5–11% narrower than the alternatives on average, with structural evidence for this effect (Section 8.2), and this advantage is consistent across prior work that includes per-operator hyperparameter tuning [24, 32].
4. Avoid  $N_3$  in either setting. The 3-task block rearrangements it adds beyond  $N_2$  are dominated on both dimensions: it is the weakest neighbourhood on solution quality and the most expensive computationally, reaching the 900 s time limit for the largest instances.

### 8.2. Theoretical Connection: Why LEX2 Narrows Intervals

Proposition 2 guarantees that reversing a non-extreme-critical arc cannot improve either bound. LEX2 specifically targets  $C_{\text{max}}^+$ , accepting moves that reduce  $C_{\text{max}}^+$  even when  $C_{\text{max}}^-$  stays put. Over many local search steps, this creates a bias toward configurations where  $G^+(\sigma)$  has a shorter critical



path — and, by the structure of interval arithmetic, a tighter bound on the overall interval width. The float analysis of Section 7.2 provides direct empirical confirmation: LEX2 solutions have more critical operations in  $G^+(\sigma)$  and less per-operation slack, while  $G^-(\sigma)$  becomes relatively looser. The asymmetry between the two extreme graphs is a structural fingerprint of the LEX2 ranking rule.

### 8.3. *Why $N_3$ Underperforms*

$N_3$  is counter-intuitive: it is a strict superset of  $N_2$  in the move-generation sense (boundary arc swaps plus three types of 3-task block rearrangements—cyclic left-shift, cyclic right-shift, and full reversal of boundary triples), yet it produces worse solutions. Each 3-task rearrangement candidate requires a full feasibility evaluation to detect cycles before it can be retained, which significantly increases the per-iteration cost. More importantly, these extra candidates do not carry improving moves in practice: the boundary arc swaps already capture the structurally meaningful transitions (as justified by Proposition 2), and the 3-task rearrangements add volume to the candidate pool without adding quality. Under best-improvement selection, a larger pool diluted with poor-quality candidates reduces the effective yield per evaluation, resulting in slower convergence to a worse plateau.

### 8.4. *Why $N_8$ Performs Well Under Tabu Search*

$N_8$  extends  $N_2$  with extra-block reinsertion moves filtered by a heads-and-tails interval estimate. Under hill climbing (Phase A), the reinsertion moves add evaluation cost without sufficient benefit: first-improvement hill climbing reaches a local optimum quickly and the larger candidate set yields few new improvements once stalled. The result is slower convergence at slightly worse quality compared with  $N_2$ , as visible in Figure 5.

Under tabu search (Phase B), this penalty reverses. Three mechanisms interact. First, the tabu list prevents revisiting recently-moved operations, forcing the search to explore the full breadth of  $N_8$ 's candidate set—including the reinsertion moves—rather than cycling near a local optimum where only arc swaps are informative. Second, tabu search employs best-improvement selection: among all candidate moves at each iteration (arc swaps and reinsertions alike), the globally best is chosen; in a larger neighbourhood this captures improvements that first-improvement hill climbing would miss. Third, the aspiration criterion allows a tabu move to be accepted if it improves the current best solution; this creates a channel through which reinsertion moves

that improve the interval criterion in aggregate (even at a mid-iteration mid-point cost) are eventually accepted.

In the deterministic setting [11], the additional reinsertion moves benefit even simple local search because there is a single critical path: every accepted move that shortens it improves the unique makespan. In the interval setting the disagreement between  $G^-$  and  $G^+$  makes candidates non-comparable more often, so the value of the larger candidate set depends on the search strategy —  $N_8$  requires a memory-based mechanism to exploit its enlarged candidate set effectively.

### 8.5. Limitations and Future Work

- Larger instances and per-operator tuning. Extending to  $\text{tai}100 \times 20$  (2000 operations) would require excessively long runtimes on the current cluster, but would be valuable: larger search spaces tend to sharpen structural differences between neighbourhoods, and the question of whether operator differences persist under per-operator tuning remains open beyond the 12 small instances studied in [24, 32].
- Other uncertainty levels. The 15% uniform interval width is a single noise level; sensitivity to other noise widths is left open.
- Hybrid algorithms.  $(\text{LEX2}, N_2)$  is the natural starting point for a dedicated uncertainty-aware metaheuristic: LEX2 directly tightens the worst-case bound (Section 8.2) and  $N_2$  provides the strongest neighbourhood structure overall;  $(\text{LEX2}, N_8)$  is a natural alternative for large instances. A dedicated algorithm that exploits the structural mechanism of Section 8.2—directly optimising moves that tighten  $G^+(\sigma)$ —could improve on the general-purpose tabu search used here.
- Other rankings. Additional admissible orders on intervals [6] could be tested with the same protocol; the structural mechanism uncovered for LEX2 should generalise to any operator that strictly prioritises one extreme.

## 9. Conclusions

I have presented a theoretical framework for neighbourhood-based local search in the Interval Job Shop Scheduling Problem grounded in the concept of *extreme critical paths* — arcs that are critical in either the best-case or

worst-case scenario graph. The resulting neighbourhood  $H(\sigma)$  satisfies feasibility, connectivity and the no-loss-of-improving-neighbours property for all four admissible interval rankings considered (EV, LEX1, LEX2, YX) simultaneously.

Five neighbourhood variants ( $N_1$ ,  $N_2$ ,  $N_3$ ,  $N_{\text{ext}}$ ,  $N_8$ ) and four ranking operators were evaluated on 82 benchmark instances of up to 1000 operations, across three phases: a  $5 \times 4$  comparison at a common parameter setting, a five-way comparison after irace-based tuning, and a comparison with three published IJSP metaheuristics.

After per-neighbourhood irace-tuned tabu search,  $N_2$  and  $N_8$  emerge as the two strongest variants—statistically tied (negligible effect  $r = 0.07$ )—while  $N_{\text{ext}}$ ,  $N_1$ , and  $N_3$  trail by medium, large, and very large effects respectively. Considered jointly with runtime,  $N_2$  is the most efficient choice across all instance sizes: it ties for best quality, is faster than  $N_{\text{ext}}$  for all instances with  $\geq 450$  operations, and is faster than  $N_8$  for small and medium instances.  $N_8$  is the preferred choice for large instances when runtimes of 200–600 s are acceptable, as it achieves the best quality on the tai50 $\times$ 15 class. Regarding ranking operators, LEX2 yields makespan intervals 5–11% narrower than the alternatives on average. A float analysis of both extreme graphs provides structural support for this finding: LEX2 solutions show more critical operations and less per-operation slack in  $G^+(\sigma)$ , consistent with LEX2 directly tightening the worst-case critical path rather than leaving more structural slack. The  $N_8$  variant’s rise to the top tier under tabu search—in contrast to its mid-table performance under hill climbing—is a key finding: extra-block reinsertion moves provide no advantage under first-improvement hill climbing (the cost exceeds the gain), but become genuinely useful under tabu search, where the memory mechanism exploits the enlarged candidate set without cycling. This underscores that the choice of local-search strategy interacts strongly with neighbourhood design in the interval setting. The comparison against three published IJSP metaheuristics (GA,  $ABC_{E3}$ , ESABC) provides external validation of the approach: the irace-tuned TS- $N_2$  achieves strictly lower average RE (%) than the previous best solver on every one of the 82 benchmark instances, with mean reductions of 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances.

These results provide actionable guidance for practitioners implementing local search for uncertain job shop scheduling: the choice of ranking operator can be used not only to optimise expected makespan but also to directly control solution uncertainty, and the choice of neighbourhood should account for

whether the problem is genuinely interval-valued or only nominally uncertain.

### **CRedit authorship contribution statement**

**Hernán Díaz Rodríguez:** Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

### **Declaration of competing interest**

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### **Data availability**

The raw experimental data underlying every figure and table in this article (run logs, aggregated CSVs, irace tuning records, instance files, and analysis scripts) are openly available on Zenodo [33]. Per-instance RE (%) results for all 70 Taillard instances are provided as a separate supplementary document [31]. The classical JSP benchmarks used to derive the interval instances are taken from the OR-Library and from [34].

### **Acknowledgements**

[TODO: Funding sources, project IDs, computing resources.]

### **References**

- [1] P. J. M. van Laarhoven, E. H. L. Aarts, J. K. Lenstra, Job shop scheduling by simulated annealing, *Operations Research* 40 (1) (1992) 113–125.
- [2] E. Nowicki, C. Smutnicki, A fast taboo search algorithm for the job shop problem, *Management Science* 42 (6) (1996) 797–813. doi:10.1287/mnsc.42.6.797.
- [3] E. Nowicki, C. Smutnicki, An advanced tabu search algorithm for the job shop problem, *Journal of Scheduling* 8 (2005) 145–159. doi:10.1007/s10951-005-6364-5.

- [4] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2009.
- [5] D. Lei, Interval job shop scheduling problems, *International Journal of Advanced Manufacturing Technology* 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.
- [6] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, *Fuzzy Sets and Systems* 220 (2013) 69–77. doi:10.1016/j.fss.2012.07.015.
- [7] S. Karmakar, A. K. Bhunia, A comparative study of different order relations of intervals, *Reliable Computing* 16 (2012) 38–72.
- [8] D. Dubois, H. Fargier, P. Fortemps, Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge, *European Journal of Operational Research* 147 (2003) 231–252.
- [9] C. Artigues, C. Briand, T. Garaix, Temporal analysis of projects under interval uncertainty, in: C. Schwindt, J. Zimmermann (Eds.), *Handbook on Project Management and Scheduling Vol. 2*, Springer, 2015, pp. 911–927. doi:10.1007/978-3-319-05915-0\_11.
- [10] I. González Rodríguez, C. R. Vela, J. Puente, R. Varela, A new local search for the job shop problem with uncertain durations, in: *Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling (ICAPS-2008)*, AAAI Press, Sydney, Australia, 2008, pp. 124–131.
- [11] J. Xie, X. Li, L. Gao, L. Gui, A new neighbourhood structure for job shop scheduling problems, *International Journal of Production Research* 61 (7) (2022) 2147–2161. doi:10.1080/00207543.2022.2053490.
- [12] Z. Xu, R. R. Yager, Some geometric aggregation operators based on intuitionistic fuzzy sets, *International Journal of General Systems* 35 (4) (2006) 417–433. doi:10.1080/03081070600574353.
- [13] I. González Rodríguez, C. R. Vela, J. Puente, A. Hernández-Arauzo, Improved local search for job shop scheduling with uncertain durations,

- in: Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS 2009), AAAI Press, Thessaloniki, Greece, 2009, pp. 154–161.
- [14] J. Puente, C. R. Vela, I. González Rodríguez, Fast local search for fuzzy job shop scheduling, in: H. Coelho, et al. (Eds.), Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010), IOS Press, 2010, pp. 739–744. doi:10.3233/978-1-60750-606-5-739.
  - [15] C. R. Vela, S. Afsar, J. J. Palacios, I. González-Rodríguez, J. Puente, Evolutionary tabu search for flexible due-date satisfaction in fuzzy job shop scheduling, *Computers & Operations Research* (2020). doi:10.1016/j.cor.2020.104931.
  - [16] P. García Gómez, J. J. Palacios, C. R. Vela, I. González-Rodríguez, Adaptive lexicographical optimisation with vague goals in green fuzzy flexible job shop scheduling, *Annals of Operations Research* (2026). doi:10.1007/s10479-026-07239-1.
  - [17] S. Afsar, J. Puente, J. J. Palacios, I. González-Rodríguez, C. R. Vela, A hybrid evolutionary approach for lexicographic green flexible job-shop with interval uncertainty, *Natural Computing* 24 (2025) 483–496. doi:10.1007/s11047-025-10016-x.
  - [18] H. Díaz, I. Díaz, I. González-Rodríguez, J. J. Palacios, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020), Vol. 1238 of Communications in Computer and Information Science, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3\_52.
  - [19] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Tardiness minimisation for job shop scheduling with interval uncertainty, in: Hybrid Artificial Intelligence Systems (HAIS 2020), Vol. 12344 of Lecture Notes in Artificial Intelligence, Springer, 2020, pp. 209–220. doi:10.1007/978-3-030-61705-9\_18.
  - [20] K. Tamssaouet, S. Dauzère-Pérès, A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems, *European Journal of Operational Research* 311 (2) (2023) 455–471. doi:10.1016/j.ejor.2023.05.018.

- [21] P. García Gómez, S. Dauzère-Pérès, C. R. Vela, I. González-Rodríguez, New bounds for move evaluation in the flexible job-shop scheduling problem, *Computers & Industrial Engineering* 212 (2026) 111696. doi:10.1016/j.cie.2025.111696.
- [22] A. Calamita, C. Meloni, M. Pranzo, M. Samà, Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning, *Flexible Services and Manufacturing Journal* (2025). doi:10.1007/s10696-025-09640-7.
- [23] H. Diaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, *Natural Computing* 22 (2023) 645–657. doi:10.1007/s11047-023-09953-2.
- [24] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, *Integrated Computer-Aided Engineering* 30 (2023) 223–242. doi:10.3233/ICA-230705.
- [25] J. Fortin, P. Zielinski, D. Dubois, H. Fargier, Criticality analysis of activity networks under interval uncertainty, *Journal of Scheduling* 13 (6) (2010) 609–627. doi:10.1007/s10951-010-0163-3.
- [26] D. Dubois, H. Fargier, J. Fortin, Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks, *Journal of Intelligent Manufacturing* 16 (2005) 407–421.
- [27] T. Garaix, C. Artigues, C. Briand, Fast minimum float computation in activity networks under interval uncertainty, *Journal of Scheduling* (2013). doi:10.1007/s10951-012-0272-2.
- [28] J. E. Beasley, OR-Library: distributing test problems by electronic mail, *Journal of the Operational Research Society* 41 (11) (1990) 1069–1072. doi:10.1057/jors.1990.166.
- [29] É. Taillard, Benchmarks for basic scheduling problems, *European Journal of Operational Research* 64 (2) (1993) 278–285. doi:10.1016/0377-2217(93)90182-M.
- [30] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, *Operations Research Perspectives* 3 (2016) 43–58. doi:10.1016/j.orp.2016.09.002.

- [31] H. Díaz Rodríguez, Supplementary material: per-instance RE (%) on Taillard IJSP instances (TA1–TA70) (2026). doi:TODO: 10.5281/zenodo.XXXXXXX.  
URL <https://zenodo.org/record/XXXXXX>
- [32] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimisation in job shop with interval uncertainty, *Logic Journal of the IGPL* (2022). doi:10.1093/jigpal/jzac016.
- [33] H. Díaz Rodríguez, Experimental dataset: Neighbourhood structures and ranking operators for the interval JSP (raw runs, configurations, instances, scripts) (2026). doi:TODO: 10.5281/zenodo.XXXXXXX.  
URL <https://zenodo.org/record/XXXXXX>
- [34] Optimizizer, Best known solutions for the Taillard job shop instances, <http://optimizizer.com/TA.php>, accessed: May 2026.